

一、技術介紹

(一)技術介紹-ChatGPT

ChatGPT 是由 OpenAI 開發的先進自然語言處理 (NLP) 模型，基於 GPT (Generative Pre-trained Transformer) 架構，旨在理解並生成類似人類的文本對話。ChatGPT 基於 Transformer 模型，核心機制是自注意力機制 (Self-Attention Mechanism) 和前饋神經網絡 (Feedforward Neural Network)。這使得模型能夠同時考慮輸入文本的不同部分，理解上下文並生成連貫的回應。ChatGPT 透過人類標註的數據訓練出獎勵模型，然後使用近端策略優化算法 (PPO, Proximal Policy Optimization) 來微調，使回應更加自然、安全、符合用戶需求。ChatGPT 能夠理解複雜的句子結構，分析語意，並產生流暢、符合上下文的回答，可幫助開發人員生成程式碼、優化代碼、修復錯誤，支援多種程式語言，可解釋概念、提供學習建議，也可用於撰寫文章、故事、詩詞、產品描述、廣告文案。

(二)限制與挑戰

ChatGPT 生成的內容可能會生成錯誤或過時的資訊，使用者使用時需進一步確認。

ChatGPT

雖然能夠記住對話中的部分內容，但長對話可能會導致信息遺失或上下文理解錯誤。模型可能反映訓練數據中的偏見，需透過監管與持續優化來減少偏見影響。對於內容準確性、上下文的記憶限制以及偏見與倫理問題待更進一步的解決。

二、問題說明

在此次作業中，我將使用 ChatGPT 來了解資訊安全常見的議題，例如 SQL Injection。了解其原理及如何防禦這種攻擊手法。

三、使用流程

請扮演一位老師 教我 SQL Injection

SQL Injection (SQL 注入) 是一種常見的網路安全漏洞，攻擊者利用應用程式對 SQL 查詢的處理不當來執行未經授權的 SQL 指令，進而訪問、修改或刪除資料庫中的資料。

作為老師，我會從基礎開始，逐步帶你理解 SQL Injection 的原理、示範攻擊方式，並教你如何防範這類攻擊。

1. SQL Injection 的基本原理

SQL Injection 主要發生在當應用程式未對使用者輸入進行適當的驗證或轉義，導致攻擊者可以透過輸入特定的 SQL 語句來操縱資料庫。

舉個例子：

假設有一個簡單的登入系統，其後端使用 SQL 來驗證帳號密碼：

```
sql
SELECT * FROM users WHERE username = 'user_input' AND password = 'pass_input';
```

如果這段程式直接把用戶輸入的內容嵌入 SQL 指令，可能會產生漏洞。

2. 基本的 SQL Injection 示範

假設某個網站的登入 SQL 語句如下：

```
sql
SELECT * FROM users WHERE username = 'admin' AND password = '12345';
```

如果沒有使用適當的防範措施，攻擊者可以輸入：

```
vbnet
Username: admin' --
Password: (隨便填)
```

這樣 SQL 查詢會變成：

```
sql
SELECT * FROM users WHERE username = 'admin' -- ' AND password = 'anything';
```

-- 是 SQL 的註解符號，這表示 AND password = 'anything' 這部分會被忽略，最終查詢只會檢查 username = 'admin'，這樣攻擊者可以繞過密碼驗證直接登入。

3. 更危險的 SQL Injection

攻擊者甚至可以嘗試獲取整個資料庫：

```
sql
' OR 1=1 --
```

這樣 SQL 查詢變成：

```
sql
SELECT * FROM users WHERE username = '' OR 1=1 -- ' AND password = '';
```

因為 1=1 永遠為真，這可能會讓攻擊者登入為第一個帳戶（通常是管理員）。

還有更進階的方式，例如：

4. 如何防範 SQL Injection

為了防範這類攻擊，應用程式應該使用：

1. 使用預備語句 (Prepared Statements)

```
python
# Python + SQLite 示例
import sqlite3
conn = sqlite3.connect('database.db')
cursor = conn.cursor()

username = input("Enter username: ")
password = input("Enter password: ")

query = "SELECT * FROM users WHERE username = ? AND password = ?"
cursor.execute(query, (username, password))
result = cursor.fetchone()
```

預備語句 (? 佔位符) 確保使用者輸入的值不會被解釋為 SQL 指令，避免 SQL Injection。

2. 使用 ORM (物件關聯映射)

如 Django 的 `QuerySet`、SQLAlchemy 這類工具，它們會自動處理 SQL 查詢，降低 SQL Injection 風險。

3. 驗證與過濾使用者輸入

- 禁止輸入特殊字元 (如 ' , " -- ;) 。
- 只允許符合規範的輸入 (如數字、特定長度的字串) 。

4. 限制資料庫權限

- 確保應用程式使用的帳戶權限最低，不能刪除或修改關鍵表格。

四、心得

本次透過ChatGPT，學習到SQL Injection是一種嚴重的安全漏洞，攻擊者可以透過惡意輸入操縱 SQL 查詢，進而竊取、篡改甚至刪除資料庫內容。這種攻擊主要發生在應用程式直接將使用者輸入拼接到 SQL 指令中，而沒有進行適當的驗證。這讓我意識到，在開發應用程式時，資料庫的安全性必須放在首要考量。最有效的防範措施是使用預備語句 (Prepared Statements)，這樣 SQL 會將使用者輸入視為變數，而不會直接執行惡意指令。此外，使用 ORM (如 Django ORM 或 Hibernate) 等工具、限制資料庫權限，也能進一步降低風險。網路安全至關重要，一個小小的漏洞可能會導致企業或個人資料洩露，甚至造成財務損失。因此，在開發應用程式時，應時刻警惕 SQL Injection 的風險，確保所有與資料庫交互的程式碼都符合安全標準，以防止潛在威脅。